

Projet THL



Abdelhak Merabtene

Num étudiant : 22406596

Dépôt GitHub :

[https://github.com/abdelMer14418/THL_
Project](https://github.com/abdelMer14418/THL_Project)

Table des matières

1. Description Générale du Projet	3
2. Arborescence du Projet	3
3. Structure de données	4
1. Non-terminaux (non_terminals).....	4
2. Terminaux (terminals)	4
3. Symbole de départ (start_symbol)	4
4. Production (productions)	4
5. Epsilon (ϵ).....	5
4. Fonctionnalités Principales	5
1. Lecture et Écriture des Grammaires	5
2. Simplification des Grammaires	6
3. Conversion en Formes Normales.....	7
4. Génération de Mots	8
5. Automatisation avec Makefile	9
6. Fonctionnement des Exécutables	9
1. Exécutable grammaire :.....	9
2. Exécutable generer :.....	10
7. Note importante	10
8. Conclusion	10

1.Description Générale du Projet

Ce projet permet de manipuler, simplifier et transformer des grammaires formelles, ainsi que de générer des mots basés sur ces grammaires. Il est structuré autour de deux scripts principaux :

- `grammaire.py` pour le traitement et la transformation des grammaires.
- `generer.py` pour la génération de mots à partir des grammaires.

Lors de l'exécution :

1. `grammaire.py` produit automatiquement deux fichiers :
 - Un fichier contenant la grammaire au format Chomsky Normal Form (extension `.chomsky`).
 - Un fichier contenant la grammaire au format Greibach Normal Form (extension `.greibach`).
2. `generer.py` génère (output simple via `printf`) les mots possibles à partir des grammaires, avec une longueur maximale spécifiée.

2.Arborescence du Projet

Voici l'organisation des fichiers et répertoires du projet :

```

/
├── .git/           # Répertoire pour la gestion de version avec Git
├── .vscode/        # Répertoire de configuration pour l'éditeur VS Code
├── build/          # Répertoire de construction (généré par PyInstaller)
├── dist/           # Répertoire contenant les exécutables générés
├── .gitignore      # Fichier pour exclure certains fichiers/répertoires dans Git
├── Abdelhak_MERABTENE.pdf # Document PDF lié au projet
├── binome.txt      # Fichier texte (peut contenir des informations sur les binômes)
├── exemple1.general # Exemple de fichier de grammaire
├── exemple1.chomsky # Grammaire convertie en CNF (Chomsky Normal Form)
├── exemple1.greibach # Grammaire convertie en GNF (Greibach Normal Form)
├── exemple2.general # Deuxième exemple de fichier de grammaire
├── exemple2.chomsky # Grammaire convertie en CNF
├── exemple2.greibach # Grammaire convertie en GNF
├── exemple3.general # Troisième exemple de fichier de grammaire
├── exemple3.chomsky # Grammaire convertie en CNF
├── exemple3.greibach # Grammaire convertie en GNF
├── generer.py       # Script Python pour générer des mots à partir des grammaires
├── generer.spec     # Fichier généré par PyInstaller pour `generer.py`
├── grammaire.py     # Script Python pour transformer et manipuler les grammaires
├── grammaire.spec   # Fichier généré par PyInstaller pour `grammaire.py`
└── Makefile        # Automatisation des tâches (compilation, exécution, tests)
  
```

3. Structure de données

Les grammaires sont représentées à l'aide de plusieurs structures de données :

1. Non-terminaux (non terminals)

- Ce sont les symboles représentant des variables dans la grammaire.
- Représentés par un ensemble Python (set).
- Exemple : S, A, B dans la règle :

```
S : aA | B
A : b | ε
```

Ici, les non-terminaux sont {S, A, B}.

2. Terminaux (terminals)

- Ce sont les symboles représentant les unités finales de la grammaire (comme les lettres d'un mot).
- Représentés également par un ensemble Python (set).
- Exemple : a, b dans la règle :

```
S : aA | B
A : b | ε
```

Ici, les terminaux sont {a, b}.

3. Symbole de départ (start symbol)

- C'est le point de départ de la grammaire (l'axiome).
- Représenté par une chaîne de caractères (str).
- Exemple :

```
S : aA | B
```

Ici, le symbole de départ est S.

4. Production (productions)

- Représentée sous forme d'un dictionnaire Python (dict) où chaque clé correspond à un non-terminal, et la valeur est une liste de règles associées.
- Exemple :

```
productions = {
    "S": ["a", "A", "B"],
    "A": ["b", "ε"],
    "B": ["a"]
}
```

Dans ce cas, $S \rightarrow aA \mid B$, $A \rightarrow b \mid \epsilon$, et $B \rightarrow a$.

5. Epsilon (ϵ)

- Représenté par la lettre E dans les fichiers d'entrée (xxx. general).
- C'est une production qui génère une chaîne vide.
- Exemple :

```
A : b | E
```

Ici, $A \rightarrow b | \epsilon$.

4. Fonctionnalités Principales

1. Lecture et Écriture des Grammaires

Le script `grammaire.py` permet de lire des grammaires dans un format textuel et de les sauvegarder après transformation :

```
def read_grammar(filename):  
  
    # Lecture des règles de grammaire à partir d'un fichier.
```

La fonction `read_grammar` permet de lire une grammaire formelle à partir d'un fichier texte et de la convertir en une structure de données manipulable dans le programme. Elle commence par initialiser plusieurs structures : un ensemble `non_terminals` pour stocker les non-terminaux, un ensemble `terminals` pré-rempli avec toutes les lettres de l'alphabet (a-z) représentant les terminaux, un dictionnaire `productions` pour associer chaque non-terminal à ses règles, et une variable `start_symbol` (axiome) pour identifier le symbole de départ de la grammaire.

Le fichier est ensuite ouvert et lu ligne par ligne. Les lignes vides ou mal formées (ne contenant pas de :) sont ignorées. Chaque ligne valide est divisée en deux parties : le non-terminal à gauche du :(left) et les règles à droite (right). Les règles sont ensuite divisées en sous-règles séparées par |, et toutes les occurrences de E (epsilon) sont remplacées par ϵ pour uniformité.

Le premier non-terminal rencontré est défini comme le symbole de départ (`start_symbol`). Chaque non-terminal (left) est ajouté à l'ensemble `non_terminals`, et ses règles sont converties en listes de symboles, puis ajoutées au dictionnaire `productions`.

Finalement, la fonction retourne quatre éléments : l'ensemble des non-terminaux (`non_terminals`), l'ensemble des terminaux (`terminals`), le symbole de départ (`start_symbol`), et le dictionnaire des productions (`productions`).

Exemple d'entrée :

```
S : aA | B  
A : b |  $\epsilon$ 
```

Sortie : Grammaire analysée sous forme d'ensembles de non-terminaux, terminaux, et règles.

2. Simplification des Grammaires

Suppression des productions nulles :

```
def remove_null Productions(non_terminals, terminals, start_symbol, productions):
    # Supprime les productions contenant ε et génère de nouvelles combinaisons.
```

La fonction `remove_null Productions` a pour objectif de supprimer les productions nulles, c'est-à-dire les règles de la forme $A \rightarrow \epsilon$ dans une grammaire. Elle commence par identifier les non-terminaux dits nullables, c'est-à-dire ceux qui peuvent produire directement ou indirectement ϵ . Pour cela, un ensemble nullable est initialisé avec les non-terminaux qui ont explicitement ϵ dans leurs règles. Ensuite, un processus itératif vérifie, pour chaque règle, si tous ses symboles appartiennent à l'ensemble nullable. Si c'est le cas, le non-terminal associé est ajouté à cet ensemble. Cette étape se poursuit jusqu'à ce qu'aucun nouveau symbole nullable ne soit détecté.

Une fois l'ensemble des symboles nullables identifié, la fonction génère de nouvelles règles en éliminant ϵ . Pour chaque production, elle crée toutes les combinaisons possibles de symboles en incluant ou excluant les symboles nullables. Par exemple, pour une règle $A \rightarrow BC$, où B est nullable, les nouvelles règles générées seront $A \rightarrow BC$ et $A \rightarrow C$. Les combinaisons invalides, comme $A \rightarrow$, sont ignorées.

En fin de compte, la fonction retourne la grammaire modifiée, où toutes les occurrences explicites de ϵ ont été éliminées, tout en maintenant la complétude des règles.

Suppression des productions unitaires :

```
def remove_unit Productions(non_terminals, productions):
    #Ce code élimine les règles du type  $A \rightarrow B$ , où  $A$  et  $B$  sont des non terminaux, en les
    #remplaçant par les règles accessibles directement depuis  $B$ .
```

La fonction `remove_unit Productions` a pour objectif de supprimer les productions unitaires dans une grammaire, c'est-à-dire les règles de la forme $A \rightarrow B$, où A et B sont des non-terminaux. Ces règles, bien qu'elles ne produisent pas directement de terminaux, peuvent être simplifiées en remplaçant $A \rightarrow B$ par les règles associées à B . Cela permet de réduire les intermédiaires inutiles dans les productions.

La fonction commence par créer un dictionnaire `new Productions`, qui contiendra les nouvelles règles sans les productions unitaires. Pour chaque non-terminal A , elle construit un ensemble `reachable`, qui regroupe tous les non-terminaux atteignables depuis A par des règles unitaires. Cela est réalisé grâce à une pile `stack`, où chaque non-terminal est exploré. Si une règle $B \rightarrow C$ est trouvée, et que C n'a pas encore été visité, il est ajouté à la pile et à l'ensemble `reachable`.

Une fois que tous les non-terminaux atteignables ont été identifiés, la fonction remplace $A \rightarrow B$ par les règles associées à B . Si une règle $B \rightarrow \alpha$ est une production non unitaire (par exemple $B \rightarrow aA$) elle est ajoutée directement à A . De cette manière, toutes les règles intermédiaires unitaires sont remplacées par leurs équivalents directs.

En résultat, la grammaire obtenue n'a plus de règles unitaires.

3. Conversion en Formes Normales

Conversion en CNF (Chomsky Normal Form) :

```
def convert_to_chomsky_nf(non_terminals, terminals, start_symbol, productions):
    # Convertit les règles en CNF.
```

La fonction **convert_to_chomsky_nf** transforme une grammaire en **Forme Normale de Chomsky (CNF)**, où chaque production respecte l'une des deux formes : $A \rightarrow BC$, où A, B, C sont des non-terminals, ou $A \rightarrow a$, où a est un **terminal**. La conversion débute par le traitement des productions existantes : si une règle contient un terminal au milieu d'une production (par exemple, $A \rightarrow aB$), un nouveau non-terminal, comme T_0 , est introduit pour représenter ce terminal ($T_0 \rightarrow a$). Si une règle contient plus de deux symboles (par exemple, $A \rightarrow BCD$), elle est décomposée en plusieurs règles intermédiaires en introduisant des non-terminals supplémentaires, tels que X_0 , pour réduire la longueur à deux symboles à chaque étape ($A \rightarrow BX_0$, $X_0 \rightarrow CD$). Les règles sont ensuite réécrites en respectant ces contraintes, et tous les nouveaux non-terminals sont ajoutés à l'ensemble des non-terminals de la grammaire. En sortie, la fonction retourne les ensembles mis à jour de non-terminals et de terminaux, le symbole de départ inchangé, et les productions restructurées. Par exemple, pour une grammaire $S: aBC$, la fonction produira $S: T_0 X_0$, $T_0: a$, $X_0: BC$, garantissant que toutes les règles respectent les contraintes de la CNF (sans oublier la longueur 2).

Conversion en GNF (Greibach Normal Form):

```
def convert_to_greibach_nf(non_terminals, terminals, start_symbol, productions):
    # Convertit les règles en GNF.
```

La fonction **convert_to_greibach_nf** transforme une grammaire en **Forme Normale de Greibach (GNF)**, où chaque production respecte la forme $A \rightarrow a\alpha$, où a est un terminal et α est une suite (éventuellement vide) de non-terminals. La transformation commence par s'assurer que chaque production démarre par un terminal. Si une production $A \rightarrow B\gamma$ commence par un non-terminal B , les règles associées à B sont substituées dans A , ce qui génère de nouvelles productions pour A . Ce processus est itératif : les règles sont réécrites jusqu'à ce qu'elles commencent toutes par un terminal. Pour gérer les dépendances cycliques ou indirectes, les substitutions sont effectuées de manière ordonnée pour éviter des boucles infinies ou des conflits.

En parallèle, les terminaux qui apparaissent seuls dans des règles sont directement conservés, et les combinaisons des non-terminals sont modifiées pour garantir que le terminal reste toujours en première position. Par exemple, une règle $S: BaC$, où $B: b|c$, sera transformée en $S: baC|caC$. Enfin, une fois toutes les substitutions effectuées, les productions résultantes sont restructurées en respectant les contraintes de la GNF.

En sortie, la fonction retourne les ensembles de non-terminals et de terminaux inchangés, le symbole de départ, et un dictionnaire de productions modifiées en GNF. Par exemple, pour une grammaire $S: AB|a$ et $A: b|c$, la fonction produira $S: bB|cB|a$, où toutes les productions démarrent par un terminal. Cette transformation est particulièrement utile pour les algorithmes de reconnaissance de chaînes basés sur des grammaires, tels que les automates à pile.

4. Génération de Mots

Le script **generer.py** génère des mots basés sur la grammaire :

```
def generate_words(grammar, start_symbol, max_length):  
    # Génère des mots jusqu'à une longueur spécifiée.
```

La fonction **generate_words** est utilisée pour générer tous les mots possibles d'une grammaire jusqu'à une longueur maximale spécifiée. Elle fonctionne de manière réursive en développant les symboles à partir du symbole de départ de la grammaire. Chaque mot est construit progressivement en remplaçant les non-terminaux par leurs productions correspondantes. Si une production contient un terminal, ce dernier est ajouté directement au mot en cours de construction. Si une production contient un non-terminal, la fonction continue à le développer en explorant toutes les productions possibles associées à ce non-terminal.

Pour gérer les mots de longueur limitée, la fonction interrompt la génération dès qu'un mot dépasse la longueur spécifiée. Une fois qu'un mot est entièrement généré (lorsqu'il ne contient plus de non-terminaux), il est ajouté à un ensemble de résultats pour éviter les doublons. Ce processus se poursuit jusqu'à ce que toutes les combinaisons possibles soient explorées.

En sortie, la fonction retourne une liste triée de tous les mots générés. Par exemple, pour une grammaire avec les règles $S: aAb$ et $A: a$, et une longueur maximale de 3, la fonction produira les mots suivants : ["a", "aa", "b"]. Cette méthode est particulièrement utile pour analyser les chaînes générées par une grammaire et pour valider les propriétés de la grammaire, comme sa capacité à générer des mots spécifiques.

Exemple d'exécution :

```
py generer.py exemple1.general 5
```

5. Automatisation avec Makefile

Compilation en exécutables indépendants avec PyInstaller :

```
$(EXECUTABLE_GRAMMAIRE): grammaire.py
    $(PYINSTALLER) --onefile grammaire.py

$(EXECUTABLE_GENERER): generer.py
    $(PYINSTALLER) --onefile generer.py
```

Pour compiler les scripts Python en exécutables indépendants, il suffit d'exécuter la commande suivante dans le terminal :

```
make
```

Exécution des tests et génération des xxx.chomsky et xxx.greibach :

```
run: $(EXECUTABLE_GRAMMAIRE) $(EXECUTABLE_GENERER)
    ./dist/grammaire exemple1.general
    ./dist/grammaire exemple2.general
    ./dist/grammaire exemple3.general
    ./dist/generer exemple1.general 5
```

6. Fonctionnement des Exécutables

Les scripts compilés en exécutables sont situés dans le répertoire dist/ :

1. Exécutable grammaire :

- Analyse exemple.general pour transformer la grammaire.
- Produit automatiquement deux fichiers :
 - exemple.chomsky : Contient la grammaire en CNF.
 - exemple.greibach : Contient la grammaire en GNF.
- Commande d'exemple :

```
./dist/grammaire exemple1.general
```

2. Exécutable generer :

- Génère les mots possibles à partir d'une grammaire.
- Commande d'exemple :

```
./dist/generer exemple1.general 5
```

7.Note importante

Pour utiliser le **Makefile**, il est impératif de s'assurer que l'outil **make** est bien installé sur le système. Sur certains OS (comme Windows), cet outil n'est pas inclus par défaut.

Personnellement, j'ai installé **MSYS2** (<https://www.msys2.org/>) et intégré son terminal dans **Visual Studio Code**, puis j'ai ajouté le chemin de **Python** et de **Make** (intégré dans MSYS2) au fichier `~/.bashrc`, afin de pouvoir exécuter la commande **make** pour mon projet Python directement depuis le terminal.

8.Conclusion

Ce projet est un outil pour manipuler des grammaires et générer des mots. Sa structure bien organisée, l'utilisation des fichiers `.general`, et la production des `xxx.chomsky` et `xxx.greibach` en font un projet facilement réutilisable. L'ajout d'une documentation utilisateur et d'une interface graphique pourrait le rendre encore plus accessible.